Local SonarQube Setup and Project Analysis Guide

Spring Boot + Maven + Java 21 + Docker

This document explains how to install SonarQube locally using Docker, create a project, generate a token, analyze a Maven/Spring Boot project, and review the results.

Step 1: Check Prerequisites

Make sure Java, Maven, and Docker are installed.

```
java -version

mvn -version

docker --version

docker ps
```

If Docker Desktop is running and these commands work, continue.

Step 2: Download the SonarQube Docker Image

```
docker pull sonarqube:community
```

Step 3: Start SonarQube

```
docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  sonarqube:community
```

Verify that the container is running:

```
docker ps
```

Step 4: Open SonarQube

Open the following address in your browser:

```
http://localhost:9000
```

For first-time login, use:

```
Username: admin
Password: Sajet@12101999
```

SonarQube may ask you to change the default password.

Step 5: Create a Local Project

In SonarQube, go to Projects → Create Project → Local Project.

Example values:

```
Project display name: Boltpe Insurance
Project key: Boltpe-Insurance
Main branch name: dev
```

Use your repository's actual main development branch. You can verify it with:

```
git branch
```

Step 6: Configure New Code

On the new-code configuration screen, choose:

```
Follows the instance's default
```

Then click Create project.

Step 7: Choose Local Analysis

When SonarQube asks how you want to analyze the project, choose:

```
Locally
```

Step 8: Generate a SonarQube Token

Create a token name such as:

```
Boltpe-Insurance-Local
```

Click Generate and copy the token immediately. It will look similar to:

```
sqp_xxxxxxxxxxxxxxxxxxxx
```

Do not commit or share this token.

You can create a new token later from:

```
My Account → Security → Tokens
```

Step 9: Go to the Project Folder

```
cd /path/to/Boltpe-Insurance
```

Confirm that pom.xml and src are present:

```
pwd
ls
```

Step 10: Build and Test the Project

```
mvn clean verify
```

Wait for BUILD SUCCESS. If JaCoCo is configured, this step also generates the coverage report.

Step 11: Check JaCoCo Coverage Report

The report is commonly generated here:

```
target/site/jacoco/jacoco.xml
```

Check it with:

```
ls target/site/jacoco
```

You may see index.html, jacoco.xml, and jacoco.csv.

Step 12: Run SonarQube Analysis

Run the following command and replace YOUR_TOKEN with your generated token:

```
mvn sonar:sonar \
  -Dsonar.projectKey=Boltpe-Insurance \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=YOUR_TOKEN
```

Successful output should include:

```
ANALYSIS SUCCESSFUL
BUILD SUCCESS
```

Step 13: Open the SonarQube Dashboard

Open SonarQube again:

```
http://localhost:9000
```

Open Projects → Boltpe Insurance. Review the following:

Quality Gate

Coverage

Reliability issues

Maintainability issues

Security issues

Security hotspots

Duplications

Code smells

Step 14: Run SonarQube Again After Code Changes

You do not need to recreate the project. Run:

```
mvn clean verify
```

```
mvn sonar:sonar \
  -Dsonar.projectKey=Boltpe-Insurance \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=YOUR_TOKEN
```

Then refresh the SonarQube dashboard.

Step 15: Stop and Start SonarQube

Stop:

```
docker stop sonarqube
```

Start later:

```
docker start sonarqube
```

Daily Workflow

After initial setup, the normal workflow is:

```
docker start sonarqube
mvn clean verify
mvn sonar:sonar -Dsonar.projectKey=Boltpe-Insurance -Dsonar.host.url=http://localhost:9000 -
Dsonar.token=YOUR_TOKEN
```

Then open http://localhost:9000 and review the updated report.

Security Note

Avoid hardcoding the token in source code or committing it to Git. You can store it temporarily as an environment variable:

```
export SONAR_TOKEN="sqp_xxxxxxxxxxxxxxxxx"
```

```
mvn sonar:sonar \
  -Dsonar.projectKey=Boltpe-Insurance \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=$SONAR_TOKEN
```